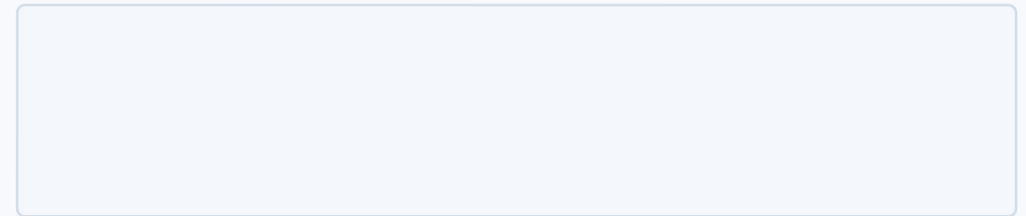


ERD → Relational Schema

Conversion Rules



ERD → Relational Schema: quick roadmap

Think in three passes: entities, attributes, then relationships.

Pass 1 — Entities

- Strong entity → one table
- Weak entity → owner PK + partial key

```
EMPLOYEE(emp_id PK, name)  
DEPENDENT(emp_id PK/FK,  
dep_name PK, dob)
```

Pass 2 — Attributes

- Composite → split into simple attributes
- Multivalued → new relation
- Derived → usually not stored

```
STUDENT(student_id PK,  
first_name, last_name)  
STUDENT_PHONE(student_id  
PK/FK, phone PK)
```

Pass 3 — Relationships

- 1:N → FK on N-side
- 1:1 → FK on one side, usually total participation side
- M:N → new junction table
- Relationship attributes go where the relationship is stored

```
TEACHER(teacher_id PK, dept_id FK)  
ENROLLMENT(student_id PK/FK,  
course_id PK/FK, semester)
```

Practice

Before converting any ERD, identify: entity keys, multivalued attributes, and exact cardinality/participation for every relationship.

Entities first

Rule 1 Strong entity → one relation (table)

Rule

- Create one table for each strong entity type.
- Put all simple attributes into that table.
- Choose one candidate key as the primary key.

Worked example

ERD entity:

Student(student_id, name, dob, email)

```
STUDENT(  
  student_id PK,  
  name,  
  dob,  
  email  
)
```

Mini-example 2

ERD entity:

Department(dept_id, dept_name, location)

```
DEPARTMENT(  
  dept_id PK,  
  dept_name,  
  location  
)
```

Practice

Practice: Convert Entity = BOOK(isbn, title, price, edition). Which attribute is the most natural PK?

Attributes mapping

Rule 2

Composite attribute → split into simple attributes

Rule

- A relational schema stores atomic columns.
- So a composite attribute is not kept as one column by default.
- Store its components instead.

Example 1

Name = {first_name, last_name}

```
STUDENT(student_id PK,  
        first_name,  
        last_name)
```

Example 2

Address = {street, city, zip}

```
CUSTOMER(customer_id  
PK,  
        street, city, zip)
```

Tip: derived attributes (such as age from date_of_birth) are usually computed, not stored.

Practice

Practice: Employee has Composite Attribute Contact = {mobile, office_phone}. Write the relation.

Attributes mapping

Rule 3 Multivalued attribute → create a separate relation

Rule

- If one entity instance can have many values for the same attribute, create a new table.
- Include the owner PK as an FK.
- Make the PK usually (owner_PK, multivalue).

Example 1: Student has many phone numbers

```
STUDENT(student_id PK, name)  
STUDENT_PHONE(student_id PK/FK,  
phone PK)
```

Example 2: Course has many textbooks

```
COURSE(course_id PK, title)  
COURSE_BOOK(course_id PK/FK,  
book_title PK)
```

Practice

Practice: Patient has multiple allergies. Build the new relation and choose its PK.

Dependent entities

Rule 4

Weak entity → owner PK + partial key

Rule

- A weak entity cannot be identified by its own attributes alone.
- Include the owner entity PK as part of the weak entity PK.
- The owner PK is also a foreign key.

Example 1

Employee owns Dependents; dependent_name is only partial key.

```
EMPLOYEE(emp_id PK, name)
DEPENDENT(emp_id PK/FK, dependent_name PK,
dob, relation)
```

Example 2

Order has weak entity OrderItem identified by line_no inside one order.

```
ORDER(order_id PK, order_date)
ORDER_ITEM(order_id PK/FK, line_no PK, qty, price)
```

Practice

Practice: LibraryCard has weak entity Fine identified by fine_no only inside one card. Convert it.

Relationship mapping

Rule 5

1 : N relationship → put the FK on the N-side

Rule

- The many-side rows need to remember which one-side row they belong to.
- So copy the one-side PK into the many-side relation as an FK.
- If the relationship has attributes, they usually stay on the many-side too.

Example 1: Department (1) — Teacher (N)

```
DEPARTMENT(dept_id PK, dept_name)  
TEACHER(teacher_id PK, teacher_name,  
dept_id FK)
```

Example 2: Customer (1) — Order (N), relationship has order_date

```
CUSTOMER(customer_id PK, name)  
ORDERS(order_id PK, order_date,  
customer_id FK)
```

Practice

Practice: One Instructor teaches many Sections. Write the SECTION relation.

Relationship mapping

Rule 6

1 : 1 relationship → place FK on one side, often the total-participation side

Rule

- A 1:1 relationship can be stored by placing one PK as an FK in the other relation.
- Prefer the side with total participation or fewer NULLs.
- Add UNIQUE to preserve one-to-one semantics.

Example 1: Person — Passport

Every Passport belongs to exactly one Person.

```
PERSON(person_id PK, name)  
PASSPORT(passport_no PK, issue_date,  
           person_id FK UNIQUE)
```

Example 2: Employee — Locker

If each Locker must belong to one Employee, keep emp_id in LOCKER.

```
LOCKER(locker_no PK, floor_no,  
        emp_id FK UNIQUE)
```

Practice

Practice: Car and Engine have a 1:1 relationship. Which side should receive the FK if every Engine must belong to a Car?

Relationship mapping

Rule 7 M : N relationship → create a new relation (junction table)

Rule

- Many-to-many cannot be stored in only one of the original relations.
- Create a new relation containing both participating PKs.
- The pair of FKs usually becomes the composite PK.

Example 1: Student enrolls in Course

```
STUDENT(student_id PK, name)  
COURSE(course_id PK, title)  
ENROLLMENT(student_id PK/FK, course_id  
PK/FK)
```

Example 2: Doctor treats Patient

```
DOCTOR(doctor_id PK, name)  
PATIENT(patient_id PK, name)  
TREATMENT(doctor_id PK/FK, patient_id PK/FK)
```

Practice

Practice: A book can be written by many authors, and an author can write many books. Build the junction relation.

Relationship attributes

Rule 8

Relationship attributes go where the relationship is stored

Rule

- For 1:N, relationship attributes usually go to the N-side relation.
- For M:N, relationship attributes go to the new junction relation.
- This keeps each fact stored exactly once.

Example 1 —

1:N Customer places Order; attribute = order_date

```
ORDERS(order_id PK, order_date,  
customer_id FK)
```

Example 2 —

M:N Student enrolls in Course; attributes = semester, grade

```
ENROLLMENT(student_id PK/FK, course_id  
PK/FK, semester, grade)
```

Practice

Practice: Employee works on Project (M:N) and the relationship has hours_worked. Where should hours_worked be stored?

Specialization / Generalization

Rule 9 ISA hierarchy → superclass table + subclass tables

Rule

- Create one table for the superclass with common attributes.
- Create one table for each subclass using the same PK.
- That subclass PK is also an FK to the superclass.

Example 1: PERSON specialized into STUDENT and TEACHER

```
PERSON(person_id PK, name, dob)
STUDENT(person_id PK/FK, program)
TEACHER(person_id PK/FK, salary)
```

Example 2: VEHICLE specialized into CAR and TRUCK

```
VEHICLE(vehicle_id PK, plate_no)
CAR(vehicle_id PK/FK, seats)
TRUCK(vehicle_id PK/FK, load_capacity)
```

Practice

Practice: Account is specialized into SavingsAccount and CurrentAccount. Write all three relations.

Fast checklist before you submit a schema

Checklist

- Did every strong entity become a table?
- Did you split composite attributes?
- Did multivalued attributes get separate relations?
- Did weak entities include owner PK + partial key?
- For 1:N, is the FK on the N-side?
- For 1:1, did you place FK on a sensible side and mark it UNIQUE?
- For M:N, did you create a junction table?
- Did relationship attributes go where the relationship is stored?
- Did you mark all PKs and FKs clearly?

Mini mixed example

ERD facts

- Department (1) — Course (N)
- Student — Course is M:N with grade
- Student has multivalued phone
- Student key = student_id

```
DEPARTMENT(dept_id PK, dept_name)
COURSE(course_id PK, title, dept_id FK)
STUDENT(student_id PK, name)
STUDENT_PHONE(student_id PK/FK, phone PK)
ENROLLMENT(student_id PK/FK, course_id PK/FK, grade)
```

Counter-question for students

Use this as a discussion / concept-check prompt.

Question

Suppose the

ERD says:

- EMPLOYEE(emp_id, email, name) where both emp_id and email are unique candidate keys.
- EMPLOYEE has multivalued skill.
- EMPLOYEE (1) — PARKING_SPOT (1) with total participation on PARKING_SPOT only.
- EMPLOYEE — PROJECT is M:N with attribute hours_worked.

Ask students to produce the full relational schema and justify:

- Which key becomes the PK in EMPLOYEE?
- How is SKILL stored?
- Which side receives the FK in the 1:1 relation, and why?
- Where does hours_worked go?

Instructor hint:

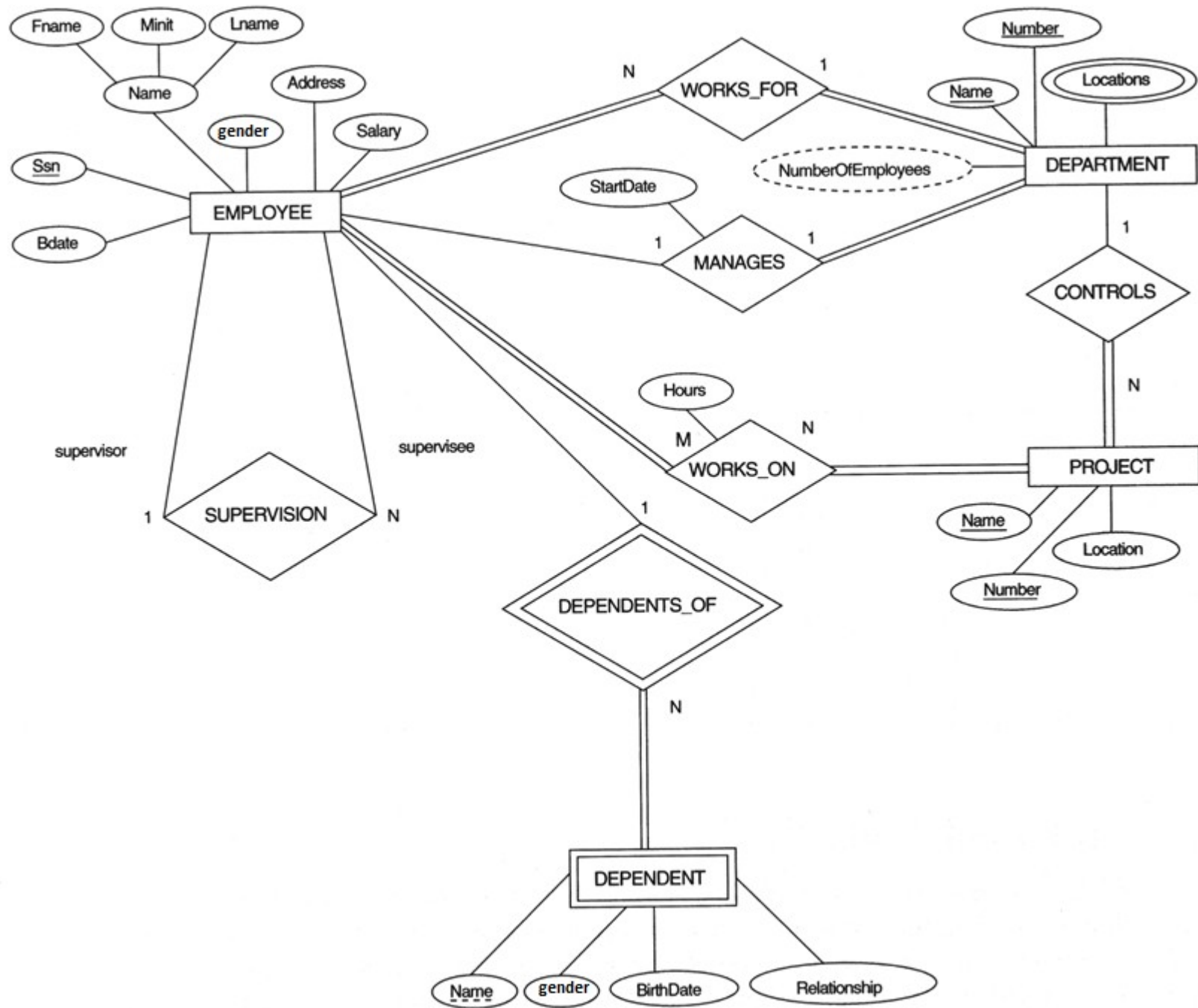
Expected ideas = pick one candidate key as PK, multivalued skill gets a new relation, FK goes into PARKING_SPOT with UNIQUE, and hours_worked belongs in the EMPLOYEE_PROJECT junction table.

2. Logical Design

Relational Model

Conversion

- ER into Relational Schema
 - We need to follow some set of rules to achieve this.
 - Step 1: Mapping of Regular/Strong Entity Types
 - Step 2: Mapping of Weak Entity Types
 - Step 3: Mapping of Binary 1:1 Relation Types
 - Step 4: Mapping of Binary 1:N Relationship Types.
 - Step 5: Mapping of Binary M:N Relationship Types.
 - Step 6: Mapping of Multivalued attributes.
 - Step 7: Mapping of N-ary Relationship Types.
- Add a **foreign key** column in the **same table** that refers to its own primary key.
- EMPLOYEE(Emp_ID, Name, Supervisor_ID)



Step 1. Convert strong entity

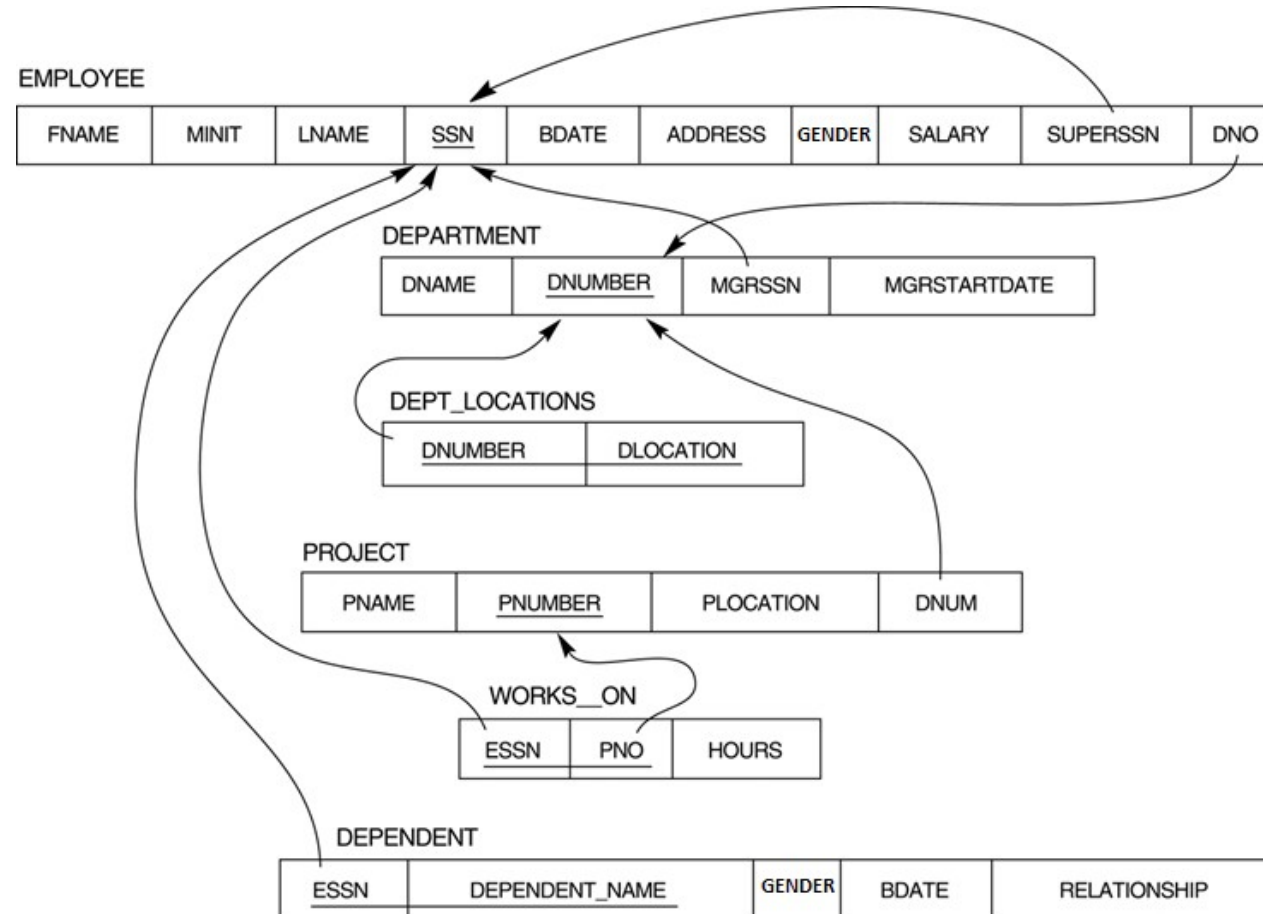
Step 2. Convert weak entity

Step 3. Convert 1:1 relationship

Step 4 Convert 1:N relationship

Step 5 Convert M:N relationship

Resultant Logical Schema



Ternary relationship types. (a) The SUPPLY relationship.

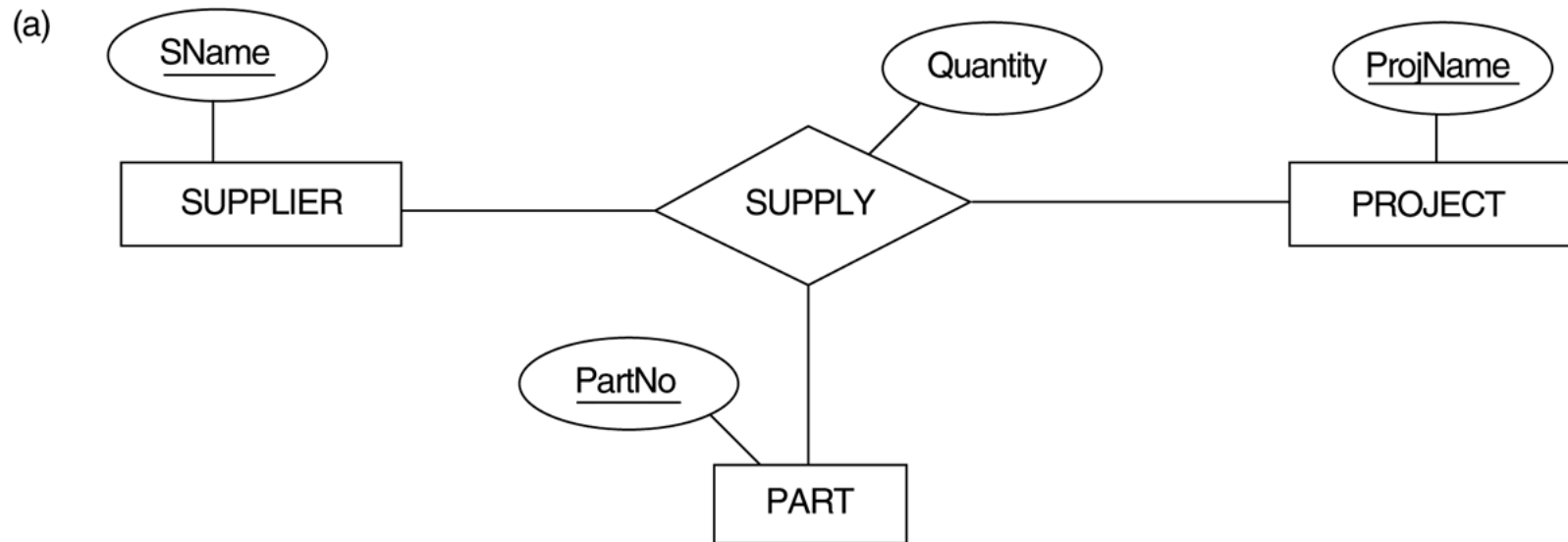


FIGURE 7.3

Mapping the n -ary relationship type SUPPLY from Figure 4.11a.

SUPPLIER

<u>SNAME</u>	...
--------------	-----

PROJECT

<u>PROJNAME</u>	...
-----------------	-----

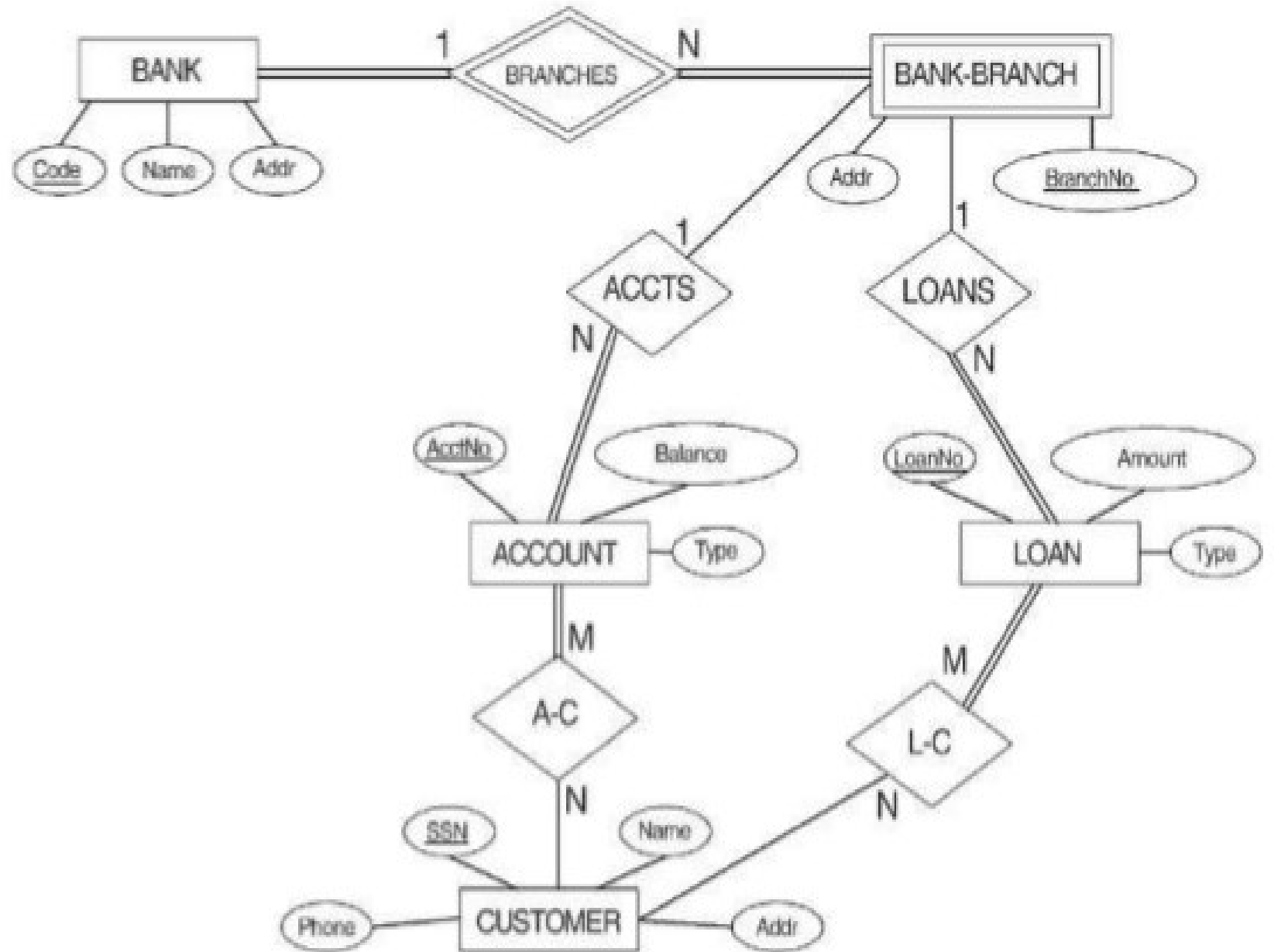
PART

<u>PARTNO</u>	...
---------------	-----

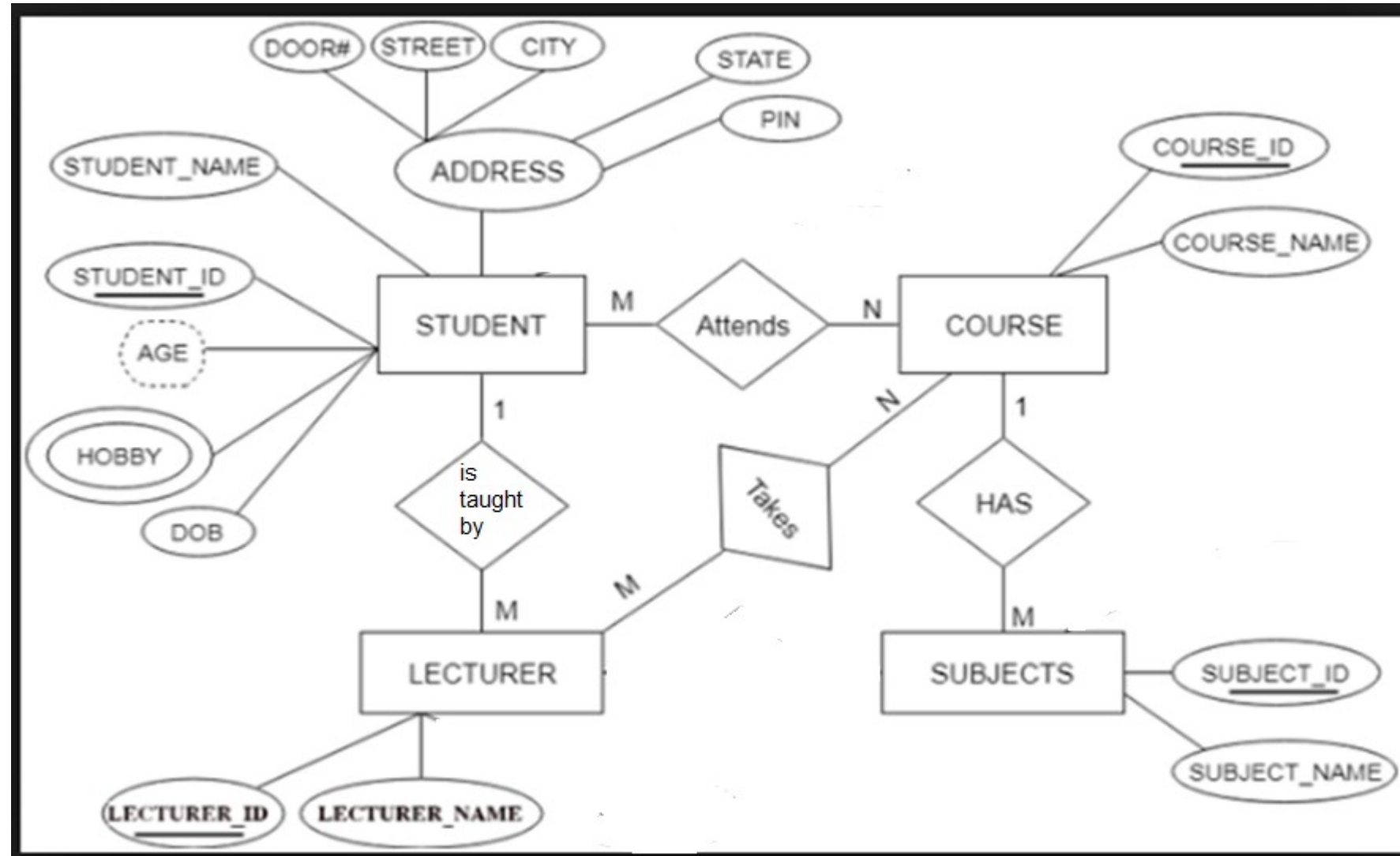
SUPPLY

<u>SNAME</u>	PROJNAME	<u>PARTNO</u>	QUANTITY
--------------	----------	---------------	----------

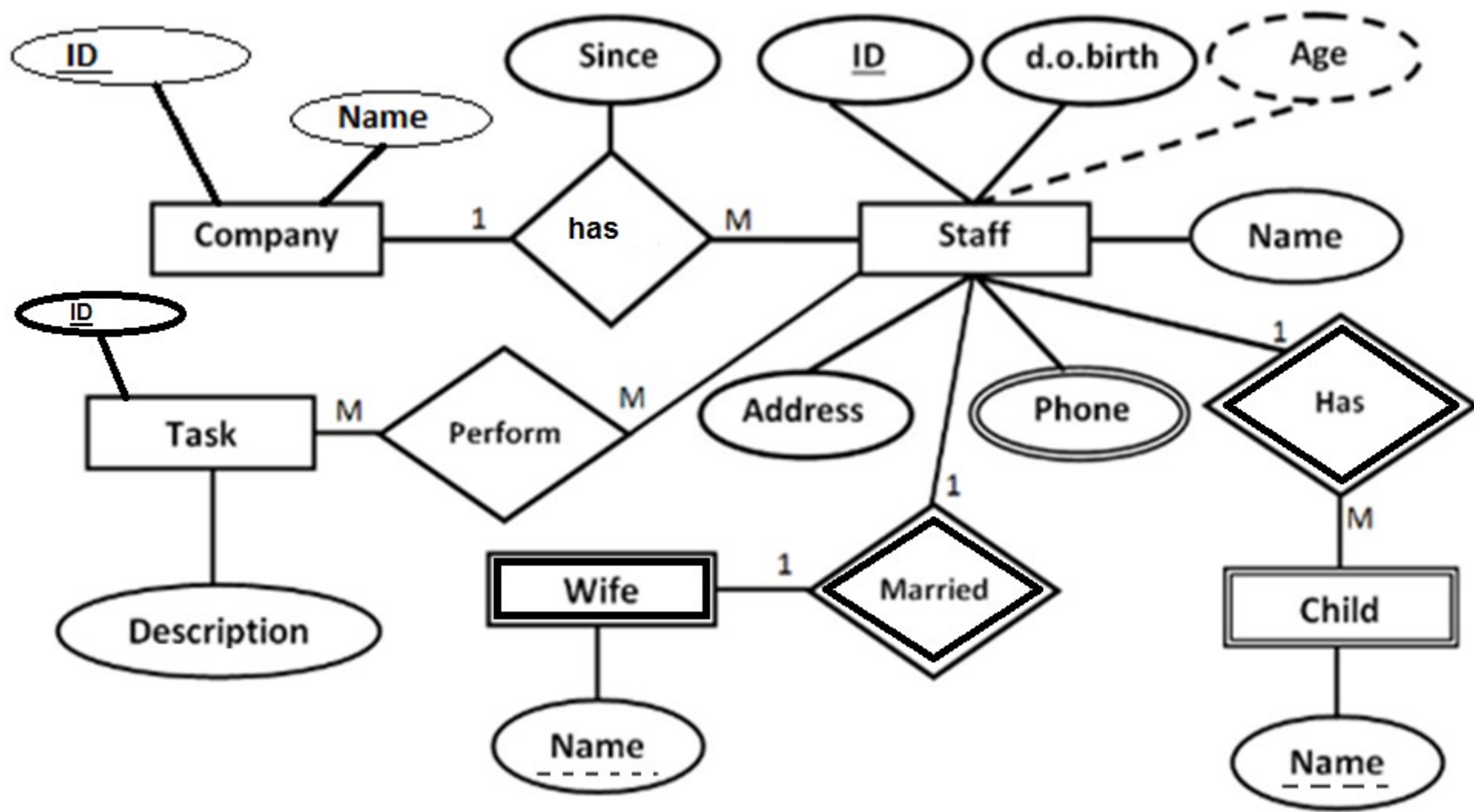
Class activi



Class Activity 2



Class Activity 3



Class Activity 4

